

Prompt Ladder: One-Layer-at-a-Time Engineering Audit (FL-07)

Author: Gautam Kumar | Workspace: small-backend/ladder | Track: AI Fluency Week 07 | Date: July 2026

1. Audit Methodology & The 'Made It Worse' Discovery

This audit evaluates prompt engineering discipline by adding exactly **one named layer at a time** across 6 runs (Baseline + 5 versions) for backend rate-limiting middleware in small-backend. Crucially, **Version 3** documents an honest **'this made it worse'** moment: adding an arbitrary line-count constraint (<25 lines) caused the AI to omit background cleanup timers, introducing a severe **infinite Map memory leak (OOM risk)**.

2. Side-by-Side Prompt Ladder Table (6 Runs Total)

Run # / Version	Layer Added	Prompt Excerpt	What Improved in Output (Result)	What Still Failed / Degraded
Run 0 Baseline	None (Weak Prompt)	"Write backend code for rate limiting."	N/A — Guessed Python/Flask instead of Node.js/Express.	Wrong language, hardcoded 10 req limit, zero RFC headers, no architecture separation.
Run 1 Version 1	Clearer Goal & Stack Context	"Write Express.js (Node.js) middleware for rate limiting a public POST route."	Stopped Python/Flask hallucinations; produced valid Node.js/Express middleware syntax.	Lazily used third-party `express-rate-limit` npm package; wrong 15-minute window.
Run 2 Version 2	Real Context & Proxy Headers	Required zero third-party packages, 10 req/60s, and proxy fallback (`x-forwarded-for`).	Eliminated npm dependency; correctly extracted proxy client IPs; implemented sliding counter.	Returned plain text string; zero RFC headers; Map grew infinitely without cleanup.
Run 3 Version 3	Line Count Trap (<25 lines) [MADE IT WORSE]	"Keep code strictly under 25 lines total; no helper functions or intervals."	Reduced line count to 11 lines and switched to basic `json({ error: ... })` body.	MADE IT WORSE: Caused infinite Map memory leak (OOM crash risk) & unreadable variables.
Run 4 Version 4	Output Format & RFC Headers	Removed brevity trap; required RFC 6585 headers (`Retry-After`), JSON error, and 5-min eviction timer.	Fixed V3 memory leak with automatic Map eviction; added RFC observability headers & clean JSON.	Lacked JSDoc annotations; no verification test suite example; hardcoded config options.
Run 5 Version 5	Few-Shot Pattern & Test Spec	Added inline JSDoc factory signature example & Jest/Supertest verification test requirements.	Gold Standard: Generated configurable factory `createRateLimiter` + automated Jest verification suite.	None — 100% testable, memory-safe, and production-ready.

3. Summary of Four Audit Notes per Version (What Actually Improved)

Version	1) What Changed in Prompt	2) What Actually Improved in Output	3) What Still Failed	4) What I Would Try Next
V1	Added framework context (`Express/Node`).	Stopped Python/Flask output; produced idiomatic Express middleware.	Lazily used third-party npm package.	Require zero external packages & add proxy IP fallback.
V2	Added in-memory Map & `x-forwarded-for` fallback.	Wrote custom Map logic and parsed multi-hop proxy IPs.	Sent plain text error; no RFC headers; no memory eviction.	Add strict line count (<25 lines) to simplify code.
V3 (Worse)	Added `<25 lines` & no intervals constraint.	Code shrunk to 11 lines; used basic JSON body.	Memory leak (OOM risk); unreadable names (`m`,`r`); no headers.	Discard brevity trap; add RFC 6585 headers & automatic eviction.
V4	Added RFC headers, JSON schema, & `.unref()` timer.	Fixed OOM leak; added RFC observability headers & retry countdowns.	Hardcoded options; lacked JSDoc & verification unit tests.	Provide few-shot factory patterns & Jest test spec.
V5	Added JSDoc export pattern & Supertest spec.	Generated configurable factory `createRateLimiter` + automated test suite.	None — 100% verified production-ready.	Package as standalone Reusable Master Prompt.

4. Reusable Engineering Master Prompt (Cleaned for Strangers)

TASK: Write a production-ready Express.js rate-limiting middleware factory `createRateLimiter(opts)` implementing an in-memory sliding window without external npm packages.

ARCHITECTURAL REQUIREMENTS: 1) Extract client IP from `x-forwarded-for`, `x-real-ip`, or `req.ip`. 2) Accept configurable options `{ windowMs = 60000, maxRequests = 10 }`. 3) Include an automatic `setInterval` cleanup timer (using `.unref()`) to delete expired IP entries every window period to prevent OOM memory leaks.

HTTP CONTRACT: On every response, set RFC 6585 headers (`Limit`, `Remaining`, `Reset`). When throttled, set `Retry-After` and return HTTP 429 with JSON: `{ "error": "RATE_LIMIT_EXCEEDED", "message": "...", "retryAfterSeconds": N }`.

DELIVERABLE: Complete ES6+ middleware file with JSDoc headers AND an accompanying Jest+Supertest verification test file asserting 200/429 status transitions.